

CI GitHub Actions : déclencheurs, jobs, commitlint

Le pipeline CI de DataShare est un seul fichier, `.github/workflows/ci.yml`, recoupé par `docs/CI.md`. Il n'y a ni Husky ni `commitlint.config.js` dans ce dépôt : le job `commitlint` s'appuie sur une action GitHub tierce, pas sur l'outil `commitlint` classique installé en local.

Déclencheurs

```
on:
  pull_request:
  push:
    branches: [master]
```

- `pull_request` sans filtre : toute PR déclenche la CI complète, quelle que soit la branche source, à chaque ouverture, réouverture ou nouveau push sur la branche de la PR.
- `push` : **seulement vers** `master`. Pousser sur une branche de travail sans PR ouverte ne déclenche rien.
- `concurrency.group: ci-${{ github.ref }}` avec `cancel-in-progress: true` : un nouveau push sur la même branche annule le run en cours, pour ne pas gaspiller de runners.
- `permissions: contents: read` par défaut : principe de moindre privilège ; seuls les jobs qui en ont besoin (`commitlint`, `security`) élèvent leurs propres permissions.

Architecture des jobs

Aucun `needs` dans le fichier : **tous les jobs tournent en parallèle**, indépendamment les uns des autres. Feedback rapide, au prix d'un peu de gaspillage si un job lourd (e2e) tourne pendant qu'un job léger (lint) échoue déjà.

Groupe	Jobs
Lint	<code>lint-front</code> , <code>lint-back</code> , <code>lint-repo</code>
Convention de commit	<code>commitlint</code>
Tests back	<code>backend-unit</code> , <code>backend-integ</code>
Tests front	<code>frontend-unit</code> , <code>frontend-integ</code> , <code>frontend-e2e</code> , <code>frontend-e2e-webkit</code> (essai, non bloquant)
Build	<code>assert-prod-bundle</code>
Sécurité	<code>security</code> (gitleaks, npm audit, Trivy)

Workflow séparé `codeql.yml` : SAST GitHub (CodeQL), sur PR, push master, et chaque lundi 04:17 UTC pour rejouer les nouvelles règles sur du code inchangé.

Le job `commitlint`

```
commitlint:
  name: commitlint
  runs-on: ubuntu-latest
  if: github.event_name == 'pull_request'
  permissions:
    pull-requests: read
  steps:
    - uses: amannn/action-semantic-pull-request@v6
      env:
        GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

Ce job ne lit **aucun commit**. Il valide le **titre de la Pull Request** au format Conventional Commits (`type(scope): description`), types autorisés : `feat`, `fix`, `docs`, `test`, `chore`, `ci`, `refactor`, `perf`, `build`, `style` .

`if: github.event_name == 'pull_request'` : le job ne tourne pas sur push master, puisqu'un push n'a pas de « titre de PR ».

Pourquoi le titre de PR et pas les commits ?

`master` impose un **merge squash uniquement** (voir plus bas). Lors d'un squash-merge, GitHub utilise le **titre de la PR comme message du commit final** sur `master` . Valider le titre de PR revient donc à valider ce qui deviendra l'unique commit sur `master` — inutile de contrôler chaque commit intermédiaire de la branche de travail, qui disparaissent au squash.

Protection de la branche `master`

D'après `docs/CI.md` :

- PR obligatoire pour merger
- Historique linéaire, pas de force-push, pas de suppression de branche
- Merge **squash uniquement**
- CI verte obligatoire (status checks requis)

Point d'attention opérationnel : après l'ajout d'un nouveau job au workflow, il faut l'ajouter à la main à la liste des *required status checks* de `master` , **une fois qu'il a tourné au moins un fois** — GitHub n'expose un nom de check dans les réglages de protection qu'après un run.

Dans DataShare

- Déclencheurs et concurrency : `ci.yml:10-17`
- Job `commitlint` : `ci.yml:87-96`
- Règles de protection et types Conventional Commits : `docs/CI.md`
- Historique réel : les commits du dépôt (`ci(security): ...` , `fix(security): ...`) suivent la convention par discipline, mais seul le titre de PR est techniquement contrôlé par la CI.

À retenir

1. `push` ne déclenche la CI que sur `master` ; toute autre branche a besoin d'une PR ouverte.
2. Tous les jobs tournent en parallèle, sans dépendances entre eux.
3. `commitlint` ne regarde pas les commits : il valide le titre de la PR via `amannn/action-semantic-pull-request` .

4. Ce choix tient au squash-merge obligatoire : le titre de PR devient le message du commit final sur `master` .
5. Un nouveau job doit être ajouté manuellement aux *required status checks* après son premier run.